

greencode — Documentação de Arquitetura de Segurança

Atividade de Avaliação Individual 1

1. Visão geral da arquitetura
2. Criptografia dos dados em repouso
 - 2.1 Algoritmo escolhido: AES-256-GCM
 - 2.2 Gerenciamento da chave mestra
 - 2.3 Escrita atômica
3. Autenticação
 - 3.1 Hash de senha: SHA-256 com salt
 - 3.2 Prevenção de enumeração de usuários
 - 3.3 Sessões e expiração por inatividade
 - 3.4 Validação de entrada em branco
4. Auditoria: journal de transações
5. Controle de acesso (RBAC)
6. Validações de regra de negócio
7. Cenários de falha testados e respostas do sistema
 - 7.1 Falhas de validação de negócio (comportamento esperado, confirmado por teste)
 - 7.2 Bugs reais encontrados durante o desenvolvimento (não hipotéticos)
 - 7.3 Metodologia de teste
8. Limitações conhecidas
9. Compatibilidade de plataforma

1. Visão geral da arquitetura

O greencode é um sistema de linha de comando (CLI) para gestão de logística reversa de equipamentos eletrônicos, escrito em Node.js com TypeScript em modo estrito (`strict: true`). A persistência é feita em arquivos de texto cifrados, sem banco de dados relacional (conforme escopo desta primeira atividade).

O código está organizado em camadas com responsabilidades bem definidas:

<code>domain/</code>	entidades e enums – regra de negócio "pura", sem I/O
<code>validators/</code>	Validador (abstrato), ValidadorCNPJ, ValidadorDataEntrada
<code>infra/</code>	criptografia, persistência em arquivo, journal de auditoria
<code>services/</code>	orquestração: cada serviço usa entidades + infra
<code>cli/</code>	parsing de comando, despacho, controle de acesso, readline
<code>config/</code>	provisionamento inicial (bootstrap do sistema)

Cada camada só conhece a camada abaixo dela — o `cli/` nunca acessa `infra/` diretamente, por exemplo, sempre passa por `services/`. Isso mantém a regra de negócio concentrada nas entidades e nos serviços, e não espalhada pela interface de linha de comando.

2. Criptografia dos dados em repouso

2.1 Algoritmo escolhido: AES-256-GCM

O enunciado exige criptografia simétrica AES-256 para todos os arquivos de persistência (credenciais, organizações, lotes, equipamentos, movimentações). A implementação usa o modo **GCM** (Galois/Counter Mode), não CBC nem ECB, por um motivo específico: GCM é um modo de **cifra autenticada** — além de manter o conteúdo em sigilo, ele detecta adulteração do arquivo cifrado. Se um byte do arquivo for alterado (por corrupção de disco ou modificação maliciosa), a operação de decifração falha explicitamente (exceção lançada), em vez de silenciosamente devolver dado corrompido ou incorreto.

Cada operação de cifragem gera um **IV (vetor de inicialização) aleatório novo de 96 bits**, nunca reaproveitado — reutilizar um IV com a mesma chave é a forma mais comum de quebrar a segurança do GCM na prática. O formato do dado cifrado gravado em disco é `iv:authTag:conteúdo` (todos em hexadecimal), autocontido: a operação inversa não precisa de nenhuma informação além da chave.

2.2 Gerenciamento da chave mestra

A chave AES-256 (32 bytes aleatórios, gerados via `crypto.randomBytes`) é criada uma única vez, no primeiro uso do sistema (**provisionamento inicial**), e gravada em `data/config.master.json`.

Limitação consciente, não descuido: esse arquivo guarda a chave em texto plano. Isso é inerente ao esquema de “chave mestra local” — não há como cifrar a própria chave com ela mesma. A segurança desse arquivo específico depende inteiramente da permissão do sistema operacional sobre ele (idealmente leitura restrita ao usuário dono do processo). É o mesmo princípio de uma chave privada SSH (`~/.ssh/id_rsa`) ou de um certificado `.pem`: a proteção vem de fora do sistema, não de dentro dele. Documentar essa decisão explicitamente é mais honesto do que fingir que o problema não existe.

2.3 Escrita atômica

Toda escrita em disco (`RepositorioArquivo`) segue o padrão “escreve em arquivo temporário, depois renomeia”:

1. Grava o conteúdo cifrado em `arquivo.json.<sufixo-aleatório>.tmp`
2. Usa `fs.renameSync` para substituir o arquivo final

Em sistemas de arquivo POSIX (Linux) e no NTFS (Windows), a operação de renomear é atômica. Isso significa que uma interrupção brusca do processo (queda de energia, `kill -9`, travamento do sistema) nunca deixa o arquivo final em estado parcialmente escrito — na pior hipótese, sobra um arquivo temporário órfão, e o arquivo de dados “de verdade” permanece com a última versão íntegra.

3. Autenticação

3.1 Hash de senha: SHA-256 com salt

O enunciado especifica hash de senha com SHA-256. A implementação usa **SHA-256 combinado com salt aleatório de 128 bits por credencial** (`hashSenha = SHA256(salt + senhaPlana)`).

Nota de transparência para a documentação (trade-off técnico): SHA-256 é um hash **rápido** — foi desenhado para computar rapidamente (é usado, por exemplo, em verificação de integridade de arquivos e em blockchain, onde velocidade é desejável). Isso o torna, em tese, mais vulnerável a ataques de força bruta do que um KDF (*key derivation function*) intencionalmente **lento**, como bcrypt, scrypt ou PBKDF2, que consomem mais tempo/memória por tentativa de propósito.

O salt aleatório por usuário já elimina o principal risco prático associado a hashes rápidos sem salt — ataques com *rainbow tables* pré-computadas — porque cada senha, mesmo repetida entre usuários, produz um hash diferente. O risco remanescente (força bruta online/offline mais rápida do que seria com um KDF lento) é uma limitação conhecida desta implementação, decorrente da escolha de algoritmo especificada pelo enunciado, e não uma recomendação de mercado. Em um sistema de produção real, a escolha recomendada seria bcrypt, scrypt ou Argon2.

A comparação de senha (`verificarSenha`) usa `crypto.timingSafeEqual`, não uma comparação direta de strings (`===`), para não vaziar informação sobre em qual posição o hash calculado diverge do hash armazenado através da diferença de tempo de resposta (*timing attack*).

3.2 Prevenção de enumeração de usuários

O fluxo de login retorna **exatamente a mesma mensagem de erro** ("Usuário ou senha inválidos.") tanto para usuário inexistente quanto para senha incorreta. Se essas duas causas fossem diferenciadas ("usuário não encontrado" vs. "senha incorreta"), um atacante poderia descobrir, por tentativa e erro, quais nomes de usuário existem no sistema (*user enumeration*) — informação que facilita ataques direcionados.

3.3 Sessões e expiração por inatividade

Cada sessão autenticada recebe um token aleatório de 256 bits (`crypto.randomBytes(32)`), não um identificador sequencial ou previsível. O enunciado exige expiração de sessão após 30 minutos de inatividade — a implementação é **deslizante** (*sliding expiration*), não um tempo fixo desde o login: a cada comando executado com sucesso, o método `validarToken` renova a expiração para 30 minutos a partir daquele momento. Uma sessão só expira se o usuário realmente ficar 30 minutos sem interagir; uma sessão de uso contínuo por horas nunca expira à toa.

Quando uma sessão expirada é detectada, o registro correspondente é removido do repositório (limpeza automática), e a CLI força novo login.

3.4 Validação de entrada em branco

Toda operação de entrada de dado (login, cadastro de usuário, senha) rejeita campos vazios ou compostos só por espaços em branco, em múltiplas camadas de defesa: na entidade (`Credencial.criar`, `verificarSenha`), no serviço (`ServicoAutenticacao`) e no parser de comando da CLI (`exigirFlag`). Essa redundância é proposital — "defesa em profundidade": mesmo que uma camada falhe ou seja contornada, as demais ainda barram a entrada inválida.

4. Auditoria: journal de transações

Toda operação que muda estado do sistema (login, logout, cadastro de organização, criação de lote, mudança de status de equipamento etc.) é registrada em um journal de transações **antes** de a mudança ser persistida no arquivo correspondente — exatamente na ordem que o enunciado exige, para permitir reconstrução do estado em caso de falha.

Cada registro (JournalTransacao) guarda: operação realizada, entidade afetada, estado anterior (dadosAntes), estado novo (dadosDepois), usuário responsável, e timestamp. **Decisão de segurança deliberada:** hashes de senha e saís **nunca** aparecem em dadosAntes/dadosDepois de operações sobre credenciais — só metadados (usuário, papel, horário) — para reduzir o que fica exposto caso o arquivo de journal seja comprometido no futuro.

O journal rotaciona automaticamente ao ultrapassar 10 MB (arquivo com timestamp no nome) e recusa apagar registros com menos de 180 dias, conforme exigido.

5. Controle de acesso (RBAC)

Os quatro papéis do enunciado (ADMINISTRADOR, OPERADOR_CADASTRO, GESTOR_ALMOXARIFADO, AUDITORIA) são verificados em um único ponto central: o Despachante da CLI. Cada comando registrado declara explicitamente quais papéis podem executá-lo; a checagem acontece antes de qualquer lógica de negócio rodar. Essa centralização evita o erro comum de esquecer a checagem de permissão em um comando novo — ela não é responsabilidade de cada comando individual, é aplicada de forma uniforme pelo despachante.

O papel AUDITORIA tem acesso somente-leitura por construção: nenhum comando de escrita (cadastrar, atualizar, criar) inclui AUDITORIA na lista de papéis permitidos.

6. Validações de regra de negócio

Regra	Onde é aplicada
CNPJ com dígito verificador válido (algoritmo oficial)	ValidadorCNPJ
CNPJ único entre organizações	ServicoOrganizacao
Data de entrada do lote: não futura, não > 90 dias no passado	ValidadorDataEntrada
Equipamento só avança para desmonte após passar por triagem	Equipamento.atualizarStatus
Justificativa obrigatória se estado físico cair 2+ categorias	Equipamento.atualizarEstadoFisico
Justificativa sempre obrigatória em mudança de status de rastreamento	Equipamento.atualizarStatus

7. Cenários de falha testados e respostas do sistema

Esta seção documenta falhas reais encontradas e corrigidas durante o desenvolvimento — não uma lista hipotética. Cada uma foi reproduzida isoladamente antes de ser corrigida.

7.1 Falhas de validação de negócio (comportamento esperado, confirmado por teste)

Cenário	Resposta do sistema
Login com usuário/senha em branco	Rejeitado antes de consultar o repositório
Login com senha incorreta	[ERRO] Usuário ou senha inválidos.
Login com usuário inexistente	Mesma mensagem acima (nunca diferenciada)
CNPJ malformatado ou com dígito verificador inválido	Rejeitado, mensagem específica do validador
CNPJ duplicado entre organizações	Rejeitado antes de persistir
Lote com data de entrada futura	Rejeitado por ValidadorDataEntrada
Lote com data de entrada > 90 dias no passado	Rejeitado por ValidadorDataEntrada
Equipamento pulando direto para EM_DESMONTE sem passar por EM_TRIAGEM	Bloqueado em Equipamento.atualizarStatus
Queda de 2+ categorias no estado físico sem justificativa	Rejeitado; com justificativa, aceito
Usuário sem papel autorizado tentando executar comando restrito	[ERRO] com o papel atual citado na mensagem
Sessão expirada (30 min de inatividade) usada em novo comando	Sessão invalidada e removida; CLI solicita novo login

7.2 Bugs reais encontrados durante o desenvolvimento (não hipotéticos)

Estes quatro problemas só apareceram ao testar o sistema **de ponta a ponta**, simulando um usuário real digitando comandos — não apareceram em testes unitários isolados por classe. Documentá-los é mais valioso do que uma lista de casos “óbvios”, porque mostra classes de erro que testes superficiais não pegam.

Bug 1 — condição de contorno na máquina de estados de triagem. A checagem que impede a 2ª fase da triagem do lote (EM_TRIAGEM → TRIAGEM_CONCLUIDA) usava um operador < onde deveria ser <=, permitindo avançar mesmo com um equipamento ainda **exatamente** no status EM_TRIAGEM (só barrava equipamentos “abaixo” dessa etapa, não “nela”). Um teste específico cobrindo esse estado intermediário exato pegou o erro; testes de “caminho feliz” não o teriam pegado. Corrigido e reconfirmado com o mesmo teste.

Bug 2 — perda de linhas de entrada no terminal (condição de corrida). Quando a entrada da CLI vem de um pipe (ou de alguém digitando rápido), o Node.js entrega todas as linhas em buffer de uma vez, de forma **síncrona**, antes mesmo de código assíncrono anterior

(como a Promise da pergunta de senha no provisionamento) ter a chance de terminar. Resultado: comandos digitados logo após a senha inicial eram silenciosamente descartados, porque chegavam a um instante em que não havia ninguém pronto para recebê-los. Corrigido com uma fila de “linhas não entregues” (`LeitorLinhaUnico`) que retém e reentrega essas linhas assim que um consumidor (nova pergunta, ou o loop de comandos) fica disponível.

Bug 3 — ordem de registro de listeners de evento. Relacionado ao anterior: a entrega das linhas em fila podia disparar, de forma síncrona, o comando `sair` (que fecha a interface de leitura) **antes** do sistema ter registrado o listener que imprime a mensagem de despedida e salva o histórico. A correção foi garantir que esse listener seja registrado antes de qualquer entrega de linha ser possível.

Bug 4 — corte de saída ao encerrar o processo. Chamar `process.exit()` imediatamente após uma escrita no console podia cortar essa escrita antes dela ser de fato entregue, quando a saída padrão está redirecionada (pipe) em vez de um terminal interativo — o Node.js bufferiza escrita em pipe de forma assíncrona. Corrigido deixando o processo encerrar naturalmente (via `process.exitCode`) em vez de forçar a saída.

7.3 Metodologia de teste

Além dos testes unitários por classe, o sistema tem um **teste de jornada completa** (`tests/jornada-completa.tests.ts`) que dirige um processo real do CLI via `stdin/stdout`, simulando desde o provisionamento inicial até a consulta de rastreabilidade de um equipamento após múltiplas movimentações — o mesmo cenário que o enunciado pede explicitamente. Esse teste foi o que revelou os quatro bugs listados acima; testes unitários isolados, por definição, não testam a interação entre componentes ao longo do tempo.

8. Limitações conhecidas

- **`config.master.json` em texto plano** (ver seção 2.2) — mitigação depende de permissão de sistema operacional, fora do controle da aplicação.
- **Hash de senha SHA-256, não um KDF lento** (ver seção 3.1) — decisão do enunciado, não recomendação de mercado.
- **Coefficiente de depreciação fixo** (20% ao ano) em `Equipamento.calcularDepreciacao` — o enunciado menciona esse valor como parâmetro configurável pelo administrador; essa configuração global ainda não foi implementada nesta primeira atividade.
- **Persistência por coleção inteira, não por registro** — `RepositorioArquivo` decifra/recifra a coleção completa a cada escrita. Correto e simples, mas não é a abordagem mais eficiente para coleções muito grandes; aceitável no escopo desta atividade.

9. Compatibilidade de plataforma

O sistema é implementado inteiramente em Node.js/TypeScript, sem dependências nativas nem chamadas de sistema específicas de plataforma (todo caminho de arquivo é construído com `path.join`, nunca concatenação manual de strings). Não há motivo técnico para incompatibilidade entre Windows 10+, Linux Ubuntu 24.04+ e derivadas — o mesmo código roda sem alteração, exigindo apenas Node.js 18+ instalado.

